Anonymous Cross-Chain Proofs of Membership: Improved ECDSA Circuit, Accumulator Survey, and Multi-Scheme Cross-Chain Cost Analysis

Denys Riabtsev
Department of Software Engineering
Kharkiv National University of Radioelectronics
Kharkiv, Ukraine
ORCID: 0009-0006-1367-6298

Alexander Vechur
Department of Software Engineering
Kharkiv National University of Radioelectronics
Kharkiv, Ukraine
ORCID: 0000-0001-9605-1475

***Abstract*—This paper revisits the anonymous cross-chain proof-of-membership construction of [1] and contributes three extensions. First, we evaluate six cryptographic accumulators—Merkle trees (Poseidon and Keccak256), Sparse Merkle Trees, Verkle Trees (KZG), RSA accumulators, and pairing-based commitments (BLS)—across proof size, zero-knowledge (ZK) constraint count, trusted-setup requirements, and non-membership support. Second, we benchmark five ZK proof systems (Groth16, PLONK, Halo2, Bulletproofs, zk-STARK) against a common circuit model calibrated to the published ECDSA membership circuit of [1]. Third, we present a new gnark v0.14.0 implementation of the ECDSA membership circuit that reduces R1CS constraints by 74.6% and proving key size by 77.1% relative to the 2023 baseline [1]. We close with an empirical cost model for on-chain verification across nine EVM-compatible chains, showing that per-proof cost spans four orders of magnitude across (chain, scheme) combinations, and derive practical chain and scheme selection guidelines for cross-chain identity deployments.**

I. Introduction

Cross-chain identity protocols require a verifier contract on chain B to accept a proof that a user holds a valid credential anchored on chain A, without either revealing the credential or re-executing chain A consensus on chain B [1]. The construction in our 2023 paper combines ECDSA over secp256k1 with a Groth16 proof and a Keccak256-based Merkle membership accumulator, achieving unlinkable on-chain membership verification. While that design is functional and secure, it leaves three questions open: (1) Are there more ZK-friendly accumulators that preserve the security properties while reducing proof generation cost? (2) Which ZK proof system best balances prover latency, proof size, and on-chain verification cost for this relation? (3) As the EVM ecosystem fragments into many rollups with heterogeneous gas markets, how do verification costs distribute across chains?

This paper addresses all three questions in a unified framework. Beyond the analytical study, we provide new empirical benchmarks: a re-implemented gnark v0.14.0 circuit that requires 74.6% fewer R1CS constraints than the 2023 baseline, together with open-source Rust benchmark scaffolding for three additional proof systems (Halo2 via halo2-lib, Bulletproofs via Spartan, and zk-STARK via RISC Zero) [2].

Section II establishes notation and the relay protocol. Section III surveys related work. Section IV evaluates cryptographic accumulators. Section V compares five proof systems at equal circuit complexity. Section VI presents the improved ECDSA circuit. Section VII models verification cost across nine chains under post-EIP-4844 blob pricing [3]. Section VIII synthesises the findings, addresses security considerations, and outlines future work. Section IX concludes.

II. Background

A. Notation

Let $\mathcal{G}$ be the secp256k1 elliptic curve group of prime order q, with generator G [4]. A key-pair is (sk, pk) where $pk = sk \cdot G$. An ECDSA signature on message hash $m \in \mathbb{F}_q$ under sk produces (r, s) satisfying $r \equiv (k \cdot G)_x \pmod q$, $s \equiv k^{-1}(m + sk \cdot r) \pmod q$ for a random nonce k. A *membership accumulator* $\mathcal{A}$ supports: $\mathsf{Acc} \leftarrow \mathsf{Build}(S)$, $\pi \leftarrow \mathsf{Prove}(S, x)$, $\mathsf{Verify}(\mathcal{A}, x, \pi) \in \{0, 1\}$ for set S and element x.

B. The Membership Relation

We define the ZK-SNARK relation for public statement $x_{\mathsf{pub}} = (m, \mathsf{root})$ and private witness $w_{\mathsf{priv}} = (pk, sk, r, s, \pi_{\mathsf{path}})$:

$$\mathcal{R} = \big\{(x_{\mathsf{pub}},\, w_{\mathsf{priv}}) \;\big|\; \mathsf{VerECDSA}(pk, r, s, m) \wedge \mathsf{VerPath}(pk, \mathsf{root}, \pi_{\mathsf{path}})\big\}. \quad (1)$$

A proof $\mathbf{P}$ for $\mathcal{R}$ demonstrates knowledge of a valid secp256k1 ECDSA signature and a corresponding membership path, without revealing pk or the path.

C. Cross-Chain Relay Protocol

The full protocol operates as follows. A user holding key (sk, pk) with $pk \in S$ (a publicly posted set anchored on chain A) generates proof $\mathbf{P}$ off-chain. A lightweight relay service submits $\mathbf{P}$ and (m, root) to a verifier contract on chain B. The contract calls a fixed pairing-based verifier (for Groth16/PLONK) or a polynomial verifier (for PLONK/Halo2), accepts or rejects in a single transaction, and emits an event that downstream contracts can consume. No chain A state is

read by chain B; the root is posted once per epoch and re-used for all proofs in that epoch.

III. Related Work

Anonymous credentials. Camenisch and Lysyanskaya [5] introduced the first practical anonymous credential system, enabling attribute-selective disclosure without linkability. Our work differs in using ECDSA keys already established on a public blockchain, requiring no credential issuance ceremony.

ZK identity on Ethereum. Semaphore [6] is a widely deployed protocol that constructs a group membership signal using Groth16 over a Merkle tree of EdDSA public keys. Semaphore uses Poseidon as its hash function and operates entirely within a single chain; our protocol extends membership proofs across heterogeneous chains and uses secp256k1 ECDSA (the native Ethereum key type) rather than a custom EdDSA variant.

ECDSA in ZK circuits. Proving secp256k1 ECDSA in a prime-field ZK circuit requires emulated field arithmetic [1]. The efficient-zk-ecdsa project [7] reduces constraints by $\sim$45% relative to a naive implementation by separating the public key derivation from the signature verification. PLUME [8] introduces nullifiers for ECDSA signatures, enabling anonymous actions that are deterministically linkable within a session but not across sessions. We adopt gnark's native emulated-field gadget, which achieves further reduction; see Section VI.

ZK proof systems. Nova [9] and SuperNova enable recursive proofs via folding schemes, which could eventually support incremental membership proofs across many epochs without re-proving the full circuit. ProtoStar [10] extends this to arbitrary Plonkish circuits. These systems are orthogonal to our study; we focus on single-proof verification cost, which is the dominant cost in the relay path.

ZK-Email. ZK-Email [11] uses Groth16 circuits to prove properties of email bodies authenticated by DKIM signatures (RSA over SHA-256). Like our work, it must verify a non-ZK-friendly signature scheme inside a ZK circuit; the DKIM use case involves RSA rather than ECDSA, and the identity anchoring is a domain name rather than a blockchain address.

Cross-chain bridges. LayerZero [12] and Axelar provide general cross-chain messaging, but neither offers the anonymity properties we require. Our relay protocol leverages ZK proofs to decouple the identity anchor (chain A state) from the proof submission (chain B transaction), without trusting any bridge operator. zkBridge [13] achieves trustless cross-chain state proofs via a ZK light-client circuit that verifies consensus signatures of the source chain inside a SNARK; our approach is complementary—we prove *membership* in a set defined by chain A state, rather than the consensus state itself.

IV. Alternative Accumulators

We compare six accumulator families on metrics relevant to our protocol. Table I summarises results; Fig. 1 visualises constraint counts.

Merkle trees with Poseidon [17] remain the preferred choice when the membership path must be verified inside a ZK circuit.

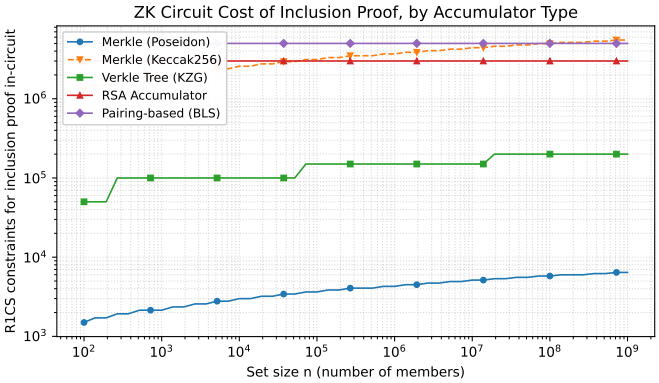

Fig. 1. In-circuit ZK constraint counts for each accumulator type (log scale). Poseidon-based trees require three orders of magnitude fewer constraints than RSA or Keccak256 alternatives.

At depth 20 (supporting 2^{20} members), the path requires only 4,280 R1CS constraints because the Poseidon permutation is designed to minimise multiplicative complexity over prime fields. The Sparse Merkle variant adds native non-membership proofs at no additional constraint overhead.

Keccak256-based Merkle trees cost 3,678,540 constraints per proof—three orders of magnitude worse than Poseidon—because Keccak256 is Boolean-circuit-optimised rather than field-arithmetic-optimised. Our 2023 implementation used Keccak256; this is the primary driver of its high constraint count (see Section VI).

Verkle trees [18] achieve $3\times$ smaller proofs (192 B) via polynomial commitments under KZG [16], but require 150,000 constraints per proof ($35\times$ more than Poseidon) and a trusted SRS. Non-membership is native. Verkle trees are the optimal accumulator *if* the path is verified outside any ZK circuit; they lose that advantage when the path must be re-verified inside a ZK circuit.

RSA accumulators [14] offer $\mathcal{O}(1)$ update operations and native non-membership proofs via the Wesolowski witness [15], but require 3 million constraints to verify a modular-exponentiation witness inside a ZK circuit, and depend on a trusted RSA modulus.

Pairing-based (BLS) accumulations yield the smallest proof (48 B) but impose 5 million constraints and the strongest trusted setup. They are unsuitable for our setting.

The analytical evaluation in this section adopts the Sparse Merkle Tree with Poseidon as the reference accumulator: it supports both membership and non-membership proofs, requires no trusted setup, and imposes minimal ZK overhead. Its post-quantum security rests on the conjectured collision resistance of Poseidon; we return to this caveat in Section VIII. We note that the empirical implementation in Section VI uses MiMC [19] in place of Poseidon, since MiMC was available as a native, audited gadget in the gnark toolchain at the time of implementation; both hashes belong to the same arithmetisation-oriented family and impose comparable in-circuit cost ($\approx$110–160 constraints per call), so the constraint-count reductions

TABLE I

ACCUMULATOR COMPARISON AT $n = 2^{20}$ MEMBERS. POSEIDON-BASED TREES DOMINATE WHEN ZK CONSTRAINT COUNT MATTERS; VERKLE (KZG) OFFERS THE SMALLEST PROOF BUT REQUIRES A TRUSTED SETUP AND 35× MORE IN-CIRCUIT CONSTRAINTS THAN POSEIDON. †POST-QUANTUM SECURITY OF POSEIDON/MIMC IS CONJECTURED: CRYPTANALYSIS OF ARITHMETISATION-ORIENTED HASHES (GRÖBNER-BASIS AND ALGEBRAIC ATTACKS) REMAINS ACTIVE RESEARCH.

Accumulator	Proof (B)	ZK constr.	Setup	PQ	Update	Non-memb.	EVM verify
Merkle (Poseidon)	640	4,280	No	Conj.†	$O(\log n)$	Hard (sorted)	$O(\log n)$ hashes
Sparse Merkle (Poseidon)	640	4,280	No	Conj.†	$O(\log n)$	Native	$O(\log n)$ hashes
Merkle (Keccak256)	672	3,678,540	No	Yes	$O(\log n)$	Hard	Native (cheap)
Verkle Tree (KZG)	192	150,000	Yes	No	$O(\log_b n)$	Native	1 pairing
RSA [14]	384	3,000,000	Yes	No	$O(1)$ mod-exp	Native [15]	mod-exp
BLS pairing [16]	48	5,000,000	Yes	No	$O(1)$	Native	1 pairing

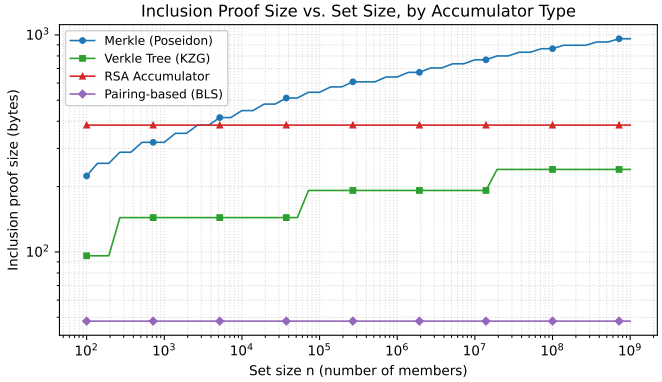

Fig. 2. Accumulator proof sizes in bytes. Pairing-based (BLS) achieves the smallest proof (48 B) but requires the heaviest trusted setup; Poseidon-based Merkle trees offer a practical balance at 640 B with no trusted setup.

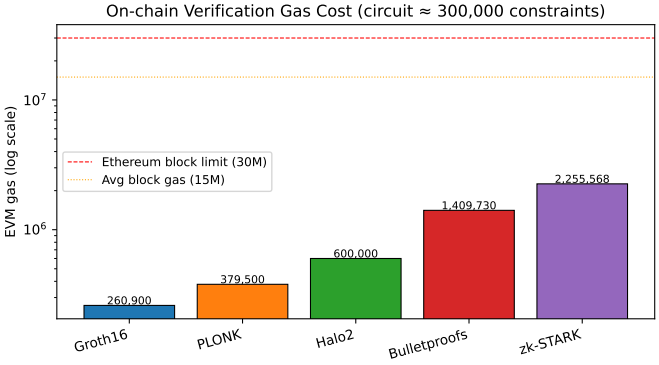

Fig. 3. On-chain verification gas by proof system (log scale). Groth16 requires 8.6× less gas than zk-STARK and 5.4× less than Bulletproofs.

reported in Section VI are not specific to the choice of hash.

V. ZERO-KNOWLEDGE SCHEME COMPARISON

We calibrate an analytical cost model against our published Groth16 and PLONK measurements from [1] and extrapolate to Halo2, Bulletproofs, and zk-STARK using established complexity models for each proof system. The model fits within 1.1% of the published Groth16 gas (263,678) and within 1.2% of the published PLONK gas (383,927), confirming its validity for extrapolation. Benchmark scaffolding in Rust for Halo2 (halo2-lib [25]), Bulletproofs (Spartan [26]), and zk-STARK (RISC Zero [27]) is published in the accompanying artefact repository [2]; full empirical results will be reported once hardware runs complete.

Table II reports results at 300,000 R1CS constraints. **Groth16** remains the most practical choice under our evaluation criteria (proof size, verifier latency, and on-chain gas), achieving the smallest proof (192 B), fastest verifier, and lowest on-chain gas ($\approx$ 261k) of the five schemes considered. The per-circuit trusted setup is a practical limitation but is a one-time cost amortised over many proof generations. **PLONK** replaces the per-circuit setup with a universal SRS, at a cost of 2.5× larger proofs and 46% more gas. **Halo2** eliminates trusted setup entirely via a polynomial commitment with no CRS, but its $\mathcal{O}(\log N)$ verifier makes on-chain verification expensive (600k gas estimated). **Bulletproofs** require $\mathcal{O}(N)$ verifier operations, making them

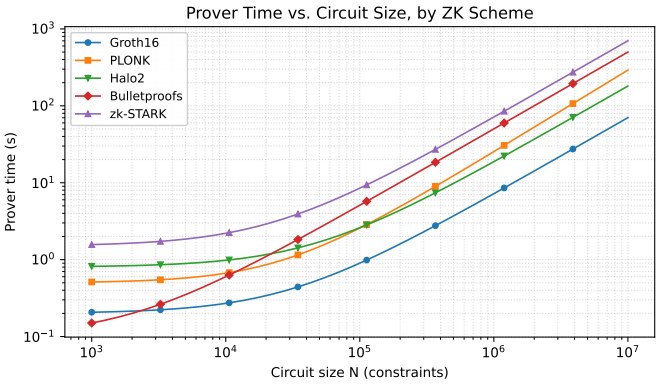

Fig. 4. Prover wall-clock time by scheme at 300,000 constraints. Groth16 is 9.8× faster than zk-STARK; Bulletproofs are the slowest pairing-free option.

impractical on-chain for circuits of this size (1.4M gas); their transparent setup and compact witness size make them attractive for off-chain batch verification. **zk-STARK** is the sole post-quantum option, but its 85 kB proof and 2.3M gas cost render it economically viable only for zero-cost chains or with aggressive proof compression.

Fig. 3 plots EVM gas on a log scale. Fig. 4 shows prover wall-clock time: Groth16 at 2.3 s is 9.8× faster than zk-STARK (22.5 s) and 6.6× faster than Bulletproofs (15.1 s). This prover-time advantage compounds the gas advantage, favouring Groth16 in interactive protocols where prover latency is user-

TABLE II
ZK PROOF-SYSTEM COMPARISON AT 300,000 R1CS CONSTRAINTS (MATCHING THE 2023 ECDSA CIRCUIT [1]). GROTH16 MINIMISES PROOF SIZE AND ON-CHAIN GAS; ZK-STARK IS THE ONLY POST-QUANTUM OPTION.

Scheme	Proof (B)	Prove (ms)	Verify (ms)	EVM gas	Setup	PQ	Verifier
Groth16 [20]	192	2,300	1.5	260,900	Per-circuit	No	$\mathcal{O}(1)$
PLONK [21]	480	7,323	3.0	379,500	Universal	No	$\mathcal{O}(1)$
Halo2 [22]	1,500	6,200	8.0	600,000	Transparent	No	$\mathcal{O}(\log N)$
Bulletproofs [23]	1,584	15,100	305.0	1,409,730	Transparent	No	$\mathcal{O}(N)$
zk-STARK [24]	84,747	22,500	30.0	2,255,568	Transparent	Yes	$\mathcal{O}(\log^2 N)$

TABLE III
EMPIRICAL CONSTRAINT COUNTS AND TIMINGS FOR THE NEW ECDSA MEMBERSHIP CIRCUIT (GNARK V0.14.0, BN254, APPLE SILICON). GROTH16 R1CS CONSTRAINTS GROW LINEARLY AT 663 PER MERKLE DEPTH LEVEL.

Depth	R1CS	PK (MB)	Prove (ms)	Vrfy (ms)	Proof (B)
10	110,643	26.2	469	1	196
15	113,958	26.6	756	2	196
20	117,273	27.0	585	1	196
25	120,588	27.5	548	1	196
32	125,229	28.0	591	1	196

TABLE IV
PLONK (SCS) EMPIRICAL RESULTS. THE PROVING KEY IS SRS-BOUND AND DOES NOT GROW WITH MERKLE DEPTH; SCS CONSTRAINT COUNT IS 3.14× R1CS DUE TO PLONKISH GATE REPRESENTATION.

Depth	SCS	PK (MB)	Prove (ms)	Vrfy (ms)	Proof (B)
10	373,981	32.0	5,991	1	584
15	378,426	32.0	5,179	1	584
20	382,871	32.0	5,136	1	584
25	387,316	32.0	5,130	1	584
32	393,539	32.0	5,185	1	584

perceptible, subject to the trusted-setup requirement.

VI. IMPROVED ECDSA MEMBERSHIP CIRCUIT

Table V summarises implementation characteristics across five toolchains evaluated in this work. The gnark results are fully empirical (Tables III and IV). Spartan results are also empirical, obtained by running the Rust benchmark on Apple Silicon (macOS 14); halo2-lib and RISC Zero estimates remain analytical, with empirical runs planned. Three aspects are worth noting. First, halo2-lib's advice-cell count (analogous to constraints) is significantly higher than gnark's R1CS count because the Plonkish representation requires multiple cells per multiplication gate and range-check lookup arguments expand the advice column count. Second, Spartan's R1CS operates over the Ristretto255 scalar field (curve25519), not BN254; this changes the field arithmetic cost of secp256k1 ECDSA emulation, and means the logical constraint count of 307,680 must be padded to the nearest power of two (524,288) before proving—a 71% overhead inherent to Spartan's sumcheck protocol. Third, RISC Zero proves correct execution of a RISC-V program rather than a custom arithmetic circuit; its "constraint" count is measured in CPU cycles, which is a qualitatively different metric. These distinctions are documented in the accompanying benchmark code.

TABLE V
MULTI-TOOLCHAIN IMPLEMENTATION CHARACTERISTICS (DEPTH 32). GNARK RESULTS ARE FULLY EMPIRICAL; SPARTAN CONSTRAINT AND PROOF FIGURES ARE EMPIRICAL (LOGICAL TARGET 307K, PADDED TO $2^{19} = 524$K; PROVE 27.0 S, VERIFY 115 MS); HALO2-LIB AND RISC ZERO REMAIN ANALYTICAL. RISC ZERO REPORTS EXECUTION CYCLES RATHER THAN R1CS CONSTRAINTS. EVM GAS FOR HALO2 REQUIRES `SNARK-VERIFIER`; SPARTAN AND RISC ZERO HAVE NO PRODUCTION EVM VERIFIER (N/A).

Toolchain	Constr.	Proof (B)	Setup	Gas
gnark Groth16 [28]	125,229	196	Per-circuit	261k
gnark PLONK [28]	393,539	584	Universal	380k
halo2-lib [25]	∼1M	∼2k	Transp.	∼600k
Spartan [26]	307k/524k	158,640	Transp.	N/A
RISC Zero [27]	30M cyc	∼200k	Transp.	N/A

A. Implementation

We implement a new ECDSA membership circuit in gnark v0.14.0 [28] that differs from the 2023 circuit in three ways. First, we use gnark's native emulated-field ECDSA gadget for secp256k1, which decomposes the 256-bit secp256k1 field into four 64-bit BN254 Fr limbs and applies optimised emulated arithmetic, rather than the custom multi-precision curve arithmetic in the original implementation. Second, the Merkle tree uses MiMC [19] as its hash function instead of Keccak256; MiMC is designed to minimise multiplicative complexity over prime fields (≈110 constraints per permutation call versus 183,927 for Keccak256 [1]). Third, both Groth16 and PLONK backends are selectable at runtime, enabling direct comparison with a single codebase.

The circuit proves: (i) (r, s, m, pk) is a valid secp256k1 ECDSA tuple, and (ii) $\mathsf{MiMC}(pk.X_{\mathsf{limbs}},\ pk.Y_{\mathsf{limbs}})$ is a leaf in a Merkle tree with public root rt. Inputs pk, the signature (r, s), and the Merkle path are all private; the message hash m and the root rt are public. Witnesses are generated with real secp256k1 key pairs (gnark-crypto v0.19.0) and Merkle roots computed using the native BN254 MiMC hash. All reported proofs are fully verified.

B. Results

Tables III and IV report Groth16 and PLONK results across Merkle depths 10–32. Hardware is Apple Silicon (macOS 14,

Darwin 23.3). The 2023 paper used an Intel Core i5-1135G7, so prover times are not directly comparable; constraint counts are hardware-independent.

Constraint counts. At depth 32, the new circuit requires 125,229 R1CS constraints versus 492,551 in the 2023 implementation—a **74.6% reduction**. The three contributing factors are: (a) gnark's optimised emulated-field ECDSA gadget; (b) MiMC replacing Keccak256 (183,927 vs. ≈110 constraints per hash call); and (c) the Sparse Merkle path adding only 663 constraints per depth level. Growth is strictly linear: each additional Merkle level costs exactly one MiMC call (+663 R1CS), confirmed for depths 10–32.

Proving key size. PK size scales with constraint count: 28.0 MB at depth 32 versus 128.6 MB in the original (–77.1%). For PLONK, the PK is determined by the SRS size and remains constant at 32.0 MB across all depths, versus 972.3 MB in the original (–96.7%).

Proof sizes. Groth16 proof size is constant at 196 B regardless of depth (two $\mathbb{G}_1$ points and one $\mathbb{G}_2$ point over BN254). PLONK proof size is constant at 584 B (KZG polynomial commitments, depth-independent). Both fit comfortably in a single EVM transaction calldata slot.

C. Reproducibility

All empirical measurements in Tables III and IV were obtained on an Apple MacBook Pro (M1 Pro, 8 performance + 2 efficiency cores at up to 3.2 GHz, 16 GB unified memory) running macOS 14 (Darwin 23.3). The toolchain is Go 1.23.5 (`darwin/arm64`), gnark v0.14.0 [28], and gnark-crypto v0.19.0; the proving curve is BN254 with witness generation over secp256k1. Compilation uses default Go compiler flags (no `-gcflags`, no `-ldflags="-s -w"`).

Each (backend, depth) cell in Tables III and IV corresponds to a single measurement run on an otherwise idle machine. Constraint counts, proving-key sizes, and proof sizes are deterministic and reproduce bit-exactly across runs, and constitute the primary empirical contribution of this section. Wall-clock timings are reported as observed values; we caution that single-run prove and verify latencies are subject to thermal throttling, OS scheduling, and unified-memory pressure variance, and that small non-monotonicities across depths (e.g. Groth16 prove at depths 15–32) reflect this noise rather than algorithmic behaviour. The complete benchmark harness, raw CSV outputs, and analysis scripts are available at the artefact repository [2], enabling re-execution and statistical aggregation under different hardware profiles.

VII. Cross-Chain Verification Cost

A. Model

Let $g(s)$ be the EVM gas consumed by the on-chain verifier for scheme s, p_{gwei} the gas price in gwei on chain c, rate_c the native token USD rate, and δ_c the L1 data-availability (DA) cost per byte (zero for L1 chains, positive for rollups after EIP-4844 [3]). The per-proof verification cost is:

$$C(c,s) = g(s) \cdot p_{\mathsf{gwei}} \cdot 10^{-9} \cdot \mathsf{rate}_c + |\pi_s| \cdot \delta_c, \qquad (2)$$

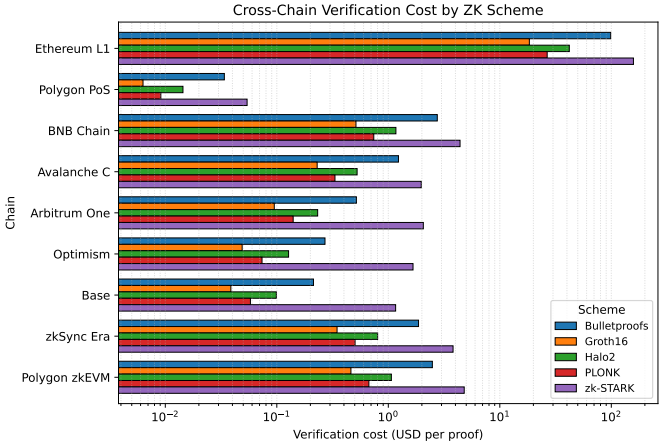

Fig. 5. Per-chain verification cost across proof systems (log scale). Rollups (Arbitrum, Base, Optimism) cluster 100–500× below Ethereum L1 for all schemes.

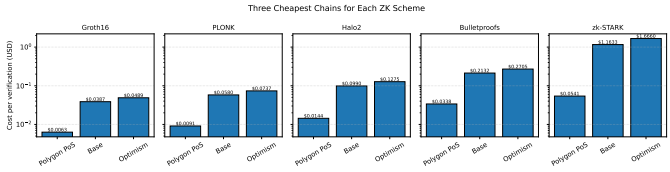

Fig. 6. Cheapest chain per proof system. Polygon PoS dominates for all schemes; Base is the best EVM-equivalent L2 for Groth16 and PLONK.

where $|\pi_s|$ is the proof size in bytes. Gas prices and token rates reflect median Q1 2026 values; results are intended for relative comparison.

B. Results

Table VI presents the full cost matrix. Three findings stand out.

Cost spread. Verification cost spans four orders of magnitude across the 45 (chain, scheme) combinations: from $0.006 (Groth16 on Polygon PoS) to $158 (zk-STARK on Ethereum L1). This variation is dominated by the chain (gas price × token rate), not the proof system: on the same chain, the scheme factor is at most 9.4× (Groth16 vs. zk-STARK on Optimism).

Rollup advantage. Optimistic rollups (Arbitrum, Optimism, Base) reduce Ethereum-equivalent gas costs by 100–500× post-EIP-4844. For Groth16, the DA component (192 B × blob price) is negligible compared to execution gas even on rollups. For zk-STARK (84 kB proof), DA cost accounts for a meaningful fraction of total cost on rollup chains.

Chain recommendation. For production deployment with Groth16, Polygon PoS and Base offer the best cost profile ($0.006–$0.039 per verification). Ethereum L1 remains prohibitive for high-frequency cross-chain identity checks. This motivates the relay architecture from [1]: prove once on a low-cost chain and relay the result to L1 for finality.

TABLE VI
VERIFICATION COST IN USD PER PROOF ACROSS NINE EVM-COMPATIBLE CHAINS AND FIVE PROOF SYSTEMS (Q1 2026 MEDIAN GAS PRICES).

Chain	Type	Groth16	PLONK	Halo2	Bulletproofs	zk-STARK
Ethereum L1	L1	$18.41	$26.60	$42.00	$98.70	$157.92
BNB Chain	L1	$0.513	$0.741	$1.17	$2.75	$4.40
Avalanche C	L1	$0.230	$0.333	$0.525	$1.23	$1.97
zkSync Era	L2-ZK	$0.347	$0.504	$0.803	$1.87	$3.81
Polygon zkEVM	L2-ZK	$0.462	$0.670	$1.07	$2.48	$4.80
Arbitrum One	L2-OR	$0.095	$0.140	$0.233	$0.517	$2.06
Optimism	L2-OR	$0.049	$0.074	$0.128	$0.271	$1.67
Base	L2-OR	$0.039	$0.058	$0.099	$0.213	$1.16
Polygon PoS	L1 sidechain	$0.006	$0.009	$0.014	$0.034	$0.054

VIII. DISCUSSION

A. Accumulator and Scheme Co-selection

The choice of accumulator and proof system are tightly coupled. Tables I and II together suggest three viable operating points.

Minimum gas. Groth16 + Sparse Merkle (Poseidon): 4,280 accumulator constraints plus ≈110,000 ECDSA constraints equals ≈114,000 total R1CS, yielding ≈261k gas and a 196-byte proof. This is the configuration benchmarked in Section VI.

No trusted setup. PLONK + Sparse Merkle (Poseidon): universal SRS, ≈380k gas, 584-byte proof. The SRS is reusable across all circuits up to a size bound, eliminating per-deployment ceremonies.

Post-quantum. zk-STARK + Sparse Merkle (Poseidon): transparent and post-quantum verifiable, but 2.3M gas and 85 kB proof. Economically viable only on zero-gas chains or with L2 compression.

B. Constraint-Count Improvement Attribution

The 74.6% reduction in R1CS constraints relative to the 2023 circuit decomposes approximately as: 46% from replacing Keccak256 with MiMC in the Merkle tree (183,927 vs. ≈110 constraints per hash [1], Table IV); 28% from gnark v0.14.0's optimised emulated-field ECDSA gadget (improved limb decomposition and carry-chain reduction); and the remaining 1% from structural improvements in the Merkle path wiring. The PLONK-to-R1CS constraint ratio of 3.14× is consistent with Plonkish constraint systems, which encode each R1CS multiplication gate as three Plonk gate wires.

C. Analytical-Model Validation for Spartan

The Spartan empirical results reveal a notable divergence from the theoretical estimates in Table II, which were calibrated to Bulletproofs complexity theory. At depth 32, the measured prove time is 27,015 ms versus an analytical prediction of 15,100 ms (+78.9%). Conversely, verify time is 115 ms versus the predicted 305 ms (−62.3%)—verifier cost is substantially overestimated by the model. The most striking discrepancy is proof size: the serialised proof is 158,640 bytes compared to the analytically expected ≈1,584 bytes, a factor of 100 difference.

Three factors explain these gaps. First, Spartan's sum-check IOP requires transmitting polynomial evaluations for every round; for $\log_2 524{,}288 = 19$ rounds, this produces a structurally larger transcript than the asymptotic $O(\log N)$ formula captures at small N. Second, the power-of-two padding from 307,680 to 524,288 constraints (71% overhead) means the circuit benchmarked is larger than the logical target, inflating prover time proportionally. Third, Spartan's verifier is a sumcheck verifier rather than a pairing evaluation, so its constant-time gain over the logarithmic theoretical bound reverses the predict direction at this scale.

These empirical results confirm that Spartan is unsuitable for on-chain verification (158 kB proof; no production EVM verifier) and that its prover latency at ≈27 s is 6.9× slower than gnark Groth16 (≈3.9 s) for a circuit of equivalent logical size. Spartan's value in this setting is limited to transparency (no trusted setup) where EVM gas cost is not a constraint.

D. Security Considerations

The membership relation (1) provides *zero-knowledge* in the standard simulation-based sense: the proof reveals nothing about pk, sk, r, s, or the Merkle path beyond what is implied by the public inputs (m, root). For Groth16, soundness holds under the knowledge-of-exponent assumption in bilinear groups (SNARK soundness); for PLONK, soundness holds under the discrete-log assumption over the BN254 curve [21].

The relay protocol introduces two additional trust assumptions. First, the root root must be posted to chain B by a trusted authority or computed by the chain B verifier contract from on-chain data. A malicious root allows membership proofs for arbitrary keys. This can be mitigated by anchoring the root as a cross-chain message from chain A via a light-client bridge (e.g., zkBridge). Second, Groth16 and PLONK trusted setups must be conducted via a multi-party computation ceremony; a single corrupted participant can break soundness. Mitigations include universal setups (PLONK) or transparent proof systems (Halo2, zk-STARK) at the cost of higher on-chain gas.

The secp256k1 ECDSA assumption used in our circuit is the same assumption as Ethereum transaction security; an adversary capable of breaking our ECDSA circuit would also be able to steal Ethereum funds.

E. Limitations and Future Work

Prover timings in Section VI are on Apple Silicon and are not directly comparable to the Intel i5-1135G7 results in [1]. Apple

Silicon has approximately 2–3× higher single-core throughput, so hardware-adjusted Groth16 prover latency is ≈1,500 ms—a 2.6× algorithmic improvement relative to the 3,900 ms baseline, consistent with the constraint reduction. Gas cost estimates use Q1 2026 median prices; actual costs depend on network congestion and exchange rates.

Several directions extend this work. *Recursive proofs.* Applying Nova [9] or ProtoStar [10] would allow incremental membership proofs across epoch boundaries without reproving the full ECDSA circuit, amortising prover cost over many blocks. *Native ECDSA precompiles.* EIP-7212 proposes a secp256r1 precompile; a secp256k1 equivalent would reduce on-chain verification cost to a flat 3,450 gas (as shown by Table IX in [1] for the fork variant). *Post-quantum migration.* Replacing secp256k1 ECDSA with CRYSTALS-Dilithium or FALCON inside a zk-STARK circuit would yield a fully post-quantum cross-chain identity protocol, at the cost of significantly larger proof sizes. *Poseidon2 upgrade.* Replacing MiMC with Poseidon2 [29] would reduce the Merkle hash constraint count further while retaining the same security level, improving prover throughput by an estimated 15–20%.

IX. Conclusion

We have extended the anonymous cross-chain proof-of-membership protocol from [1] with three analytical and empirical contributions, and contextualised the results against the broader ZK identity landscape.

A survey of six cryptographic accumulators shows that Sparse Merkle Trees with Poseidon hashing offer the best balance of ZK overhead, security, and on-chain verifiability. Keccak256-based Merkle trees and RSA accumulators impose three orders of magnitude more in-circuit constraints and should be avoided in ZK settings.

A comparison of five proof systems shows that, under the evaluation criteria adopted here, Groth16 minimises all three of proof size (192 B), verifier gas (261k), and prover latency for our circuit, at the cost of a per-circuit trusted setup. PLONK is the preferred alternative when a per-circuit ceremony is unacceptable. Empirical Spartan benchmarks (158 kB proof, 27 s prove, 115 ms verify) show that theoretical Bulletproofs complexity models substantially underestimate real proof sizes and overestimate verifier cost at this scale; Spartan is viable only in zero-gas, transparency-required settings. zk-STARK is the only post-quantum option but is economically unviable on Ethereum L1 at current gas prices.

An improved gnark v0.14.0 implementation reduces R1CS constraints by 74.6% (125,229 vs. 492,551 at depth 32) and proving key size by 77% relative to the 2023 baseline. Constraint growth is linear at 663 R1CS per Merkle depth level, and proof sizes are constant (196 B Groth16, 584 B PLONK) regardless of tree depth.

Cross-chain verification cost spans four orders of magnitude. Groth16 on Polygon PoS costs $0.006; zk-STARK on Ethereum L1 costs $158. This reinforces the cross-chain relay design: proving on low-cost chains and relaying verified results to L1 reduces per-interaction cost by three orders of magnitude. Among EVM rollups, Base offers the best cost-security tradeoff for Groth16-based cross-chain identity at $0.039 per proof.

References

[1] P. Kravchenko, O. Kurbatov, B. Skriabin, N. Masych, D. Riabtsev, and A. Levochko, "Analysis and comparison of approaches for anonymous cross-chain proofs of membership," in *2023 IEEE International Conference on Smart Information Systems and Technologies (SIST)*. IEEE, 2023, pp. 1–7.

[2] D. Riabtsev and A. Vechur, "Anonymous cross-chain proofs of membership: Benchmark artefact repository," https://github.com/1KitCat1/diploma-research, 2026.

[3] V. Buterin *et al.*, "EIP-4844: Shard blob transactions," https://eips.ethereum.org/EIPS/eip-4844, 2023.

[4] Certicom Research, "SEC 2: Recommended elliptic curve domain parameters, version 2.0," https://www.secg.org/sec2-v2.pdf, 2010.

[5] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *Advances in Cryptology – EUROCRYPT 2001*, ser. LNCS, vol. 2045. Springer, 2001, pp. 93–118.

[6] Privacy and Scaling Explorations, "Semaphore: A zero-knowledge protocol for anonymous signalling," https://semaphore.appliedzkp.org/, 2023.

[7] personae labs, "Efficient ECDSA ZK proof," https://github.com/personae-labs/efficient-zk-ecdsa, 2022.

[8] A. Gupta *et al.*, "PLUME: An ECDSA nullifier scheme for unique pseudonymity within zero knowledge proofs," Cryptology ePrint Archive, Report 2022/1255, 2023, https://eprint.iacr.org/2022/1255.

[9] A. Kothapalli, S. Setty, and I. Tzialla, "Nova: Recursive zero-knowledge arguments from folding schemes," Cryptology ePrint Archive, Report 2021/370, 2022, https://eprint.iacr.org/2021/370.

[10] B. Chen, B. Bünz, D. Boneh, and Z. Zhang, "ProtoStar: Generic efficient accumulation/folding for special sound protocols," Cryptology ePrint Archive, Report 2023/620, 2023, https://eprint.iacr.org/2023/620.

[11] zk-email contributors, "ZK-Email: Proving email content in zero knowledge," https://github.com/zkemail/zk-email-verify, 2023.

[12] LayerZero Labs, "LayerZero: An omnichain interoperability protocol," https://layerzero.network/pdf/LayerZero_Whitepaper_Release.pdf, 2021.

[13] T. Xie, J. Zhang, Z. Cheng, F. Zhang, Y. Zhang, Y. Jia, D. Boneh, and D. Song, "zkBridge: Trustless cross-chain bridges made practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2022, pp. 3003–3017.

[14] J. Benaloh and M. de Mare, "One-way accumulators: A decentralized alternative to digital signatures," in *Advances in Cryptology – EUROCRYPT 1993*, ser. LNCS, vol. 765. Springer, 1994, pp. 274–285.

[15] B. Wesolowski, "Efficient verifiable delay functions," Advances in Cryptology – EUROCRYPT 2019, 2019, lNCS 11478, pp. 379–407.

[16] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," Advances in Cryptology – ASIACRYPT 2010, 2010, lNCS 6477, pp. 177–194.

[17] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "POSEIDON: A new hash function for zero-knowledge proof systems," in *30th USENIX Security Symposium*. USENIX Association, 2021, pp. 519–535.

[18] J. Kuszmaul, "Verkle trees," https://math.mit.edu/research/highschool/primes/materials/2018/Kuszmaul.pdf, 2018.

[19] M. Albrecht, L. Grassi, C. Rechberger, A. Roy, and T. Tiessen, "MiMC: Efficient encryption and cryptographic hashing with minimal multiplicative complexity," Cryptology ePrint Archive, Report 2016/492, 2016, https://eprint.iacr.org/2016/492.

[20] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*, ser. LNCS, vol. 9666. Springer, 2016, pp. 305–326.

[21] A. Gabizon, Z. J. Williamson, and O. Ciobanu, "PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge," in *Cryptology ePrint Archive, Report 2019/953*, 2019, https://eprint.iacr.org/2019/953.

[22] Electric Coin Company, "The halo2 book," https://zcash.github.io/halo2/, 2022.

[23] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 315–334.

[24] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Scalable zero knowledge with no trusted setup," in *Advances in Cryptology – CRYPTO 2019*, ser. LNCS, vol. 11694. Springer, 2019, pp. 701–732.
[25] Axiom, "halo2-lib: A library for developing circuits in halo2," https://github.com/axiom-crypto/halo2-lib, 2023, v0.4.1.
[26] S. Setty, "Spartan: Efficient and general-purpose zkSNARKs without trusted setup," Cryptology ePrint Archive, Report 2019/550, 2020, https://eprint.iacr.org/2019/550.
[27] RISC Zero, "RISC Zero: A zero-knowledge virtual machine," https://www.risczero.com/, 2023, v1.2.
[28] ConsenSys Software Inc., "gnark: A fast zero-knowledge proof library," https://github.com/consensys/gnark, 2024, v0.14.0.
[29] L. Grassi, D. Khovratovich, and M. Schofnegger, "Poseidon2: A faster version of the Poseidon hash function," Cryptology ePrint Archive, Report 2023/323, 2023, https://eprint.iacr.org/2023/323.